

안녕하세요,

체계를 세우고 반복을 자동화하는 SW QA 엔지니어 김상옥입니다.

표준이 있는 곳에선 정착시키고, 없는 곳에선 만듭니다. 자동차 임베디드에서 시작해 공공·교육 웹을 거쳐, 지금은 산업 AI 비전 검수 플랫폼 **Daisy**의 품질을 책임지고 있습니다. 도메인은 세 번 바뀌었지만 들어가서 한 일의 순서는 늘 같았습니다 — **체계를 세우고, 반복을 자동화하고, 프로세스로 풀리지 않는 것은 도구를 직접 만들어 풀었습니다.**



010-4482-2033



kx2471@gmail.com



github.com/kx2471

ABOUT ME

어떻게 일하는 사람인가

정형화된 표준 아래에서 대규모 검증을 운영해봤고, 표준이 없는 곳에서는 직접 만들어봤습니다. **두 경험을 모두 가진 것이 제 강점입니다.**



프로세스 설계 — 정착과 구축

ASPICE Level 1 SWE.6 인증 환경에서의 **구조적 QA 수행**과, 프로세스 부재 환경에서 이슈 트래커 워크플로우 · Git 브랜치 전략 · 자동화 규칙을 **0에서 설계**한 경험을 모두 보유.



테스트 자동화 · 회귀 검증

Playwright · Node.js 기반 MLOps E2E 회귀 시스템(실서버 3-worker 병렬) 1인 설계 · 구축·운영, 팀 **셀프서비스 실행 러너 구축**, Selenium UI 자동화, Postman API 검증, Jenkins · GitHub Actions 연계.



자동차 임베디드 검증

HILs 기반 IVI 시스템 검증, CAN 통신 · CANoe 기반 네트워크 검증, 자율주행 제어 애플리케이션 검증, **KOLAS 공인 신뢰성 검사 통과.**



웹 품질 · 보안 점검

KRDS 웹 접근성 검증, REST API 검증 과정에서 **서버 측 검증 로직 부재를 지적하고 개선 주도**, Burp Suite · Lighthouse 기반 웹 보안·품질 진단.



AI 활용 · 도구 직접 제작

Claude Agent 기반 Regression TC Creator, Slack + AWS Lambda 기반 QA Request 봇, Google Apps Script 기반 QA Test Run 관리 도구, OCR 매크로 · KPI Calculator 등 실무 유틸리티 직접 개발.



협업 · 문서화

Jira · Xray · Jira Automation · ClickUp · Confluence 기반 테스트 관리, 애자일 스크럼 운영, 월간/분기 품질현황보고 경영진 보고 경험.

일하는 순서 — ① 체계를 세운다: 표준이 있으면 정착시키고, 없으면 만든다 · ② 반복을 자동화한다: 한 번 손으로 한 검증은 다시 손으로 하지 않는다 · ③ 필요한 도구를 직접 만든다: 프로세스로 안 풀리면 코드로 푼다

SKILLS

기술 스택

실무에서 직접 쓴 것만 적었습니다.

테스트 관리 · 협업

Jira

Jira Xray

Jira Automation

ClickUp

Confluence

Slack

Notion

테스트 자동화 · 스크립팅

Playwright

Selenium

Postman

Python

Node.js

Shell Script

Jenkins

GitHub Actions

AI · LLM 활용

Claude Agent

MCP (ClickUp · Google Sheets)

LLM 응답 정확도 검증

자동차 HILs · 진단

Vector CANoe

CANalyzer

Monaco

ETFW

DCM 연동

웹 보안 · 품질 진단

Burp Suite

Lighthouse

KRDS 웹 접근성

로그 분석 · 모니터링

DLT Viewer

Android ADB

커스텀 로그 필터

서버리스 · 백엔드 연동

AWS Lambda

Slack API

ClickUp API

Google Sheets API

Google Apps Script

검증 환경 · 플랫폼

Linux 기반 IVI

HILs 장비

실차 테스트

Web / Mobile

AI/ML Vision Inspection

LLM 기반 시스템

기타

Git

GitHub

Figma

TypeScript

WORK

대표 작업

각 항목은 문제 → 한 일 → 결과로 정리했고, 설계 판단과 실제 운영 화면은 상세 페이지에 담았습니다. (민감

01 · PROCESS DESIGN

QA 프로세스를 0에서 1로

이슈 관리 체계가 없던 환경에 워크플로우 · Priority 기준 · Root Cause 의무화 · 배포 게이트 · RC 검증 절차를 설계하고, 규칙이 실제로 동작하도록 자동화까지 붙였습니다.

워크플로우 설계

QA Request 봇

배포 게이트

RC 검증

Test Run 도구

RCA

[자세히 보기 →](#)

02 · TEST AUTOMATION

MLOps 파이프라인 무인 회귀 검증

실서버 · 단일 공유 계정 · 고비용 학습이라는 제약 위에서 테스트가 서로를 깨뜨리지 않는 구조를 설계해, 하루 이상 걸리던 릴리스 검증을 명령 한 줄로 바꿨습니다.

Playwright

3-worker 병렬

픽스처 재사용

트러블슈팅

Slack 리포터

[자세히 보기 →](#)

03 · QA ENABLEMENT

테스트 러너 — 모두가 검증할 수 있는 환경

전사가 같은 제품을 만드는데 그 제품을 검증할 수 있는 사람은 몇 명뿐이었습니다. 만드는 사람이라면 누구나 자기가 바꾼 것을 직접 확인할 수 있어야 한다고 보고, 설치도 명령어도 없이 브라우저만 있으면 검증이 돌아가는 환경을 만들었습니다.

품질 책임의 분산

QA 이네이블먼트

셀프서비스 UI

안전한 기본값

의존성 자동 해결

사용자 문서화

[자세히 보기 →](#)

04 · AI IN WORKFLOW

AI Agent — Regression TC Creator

제가 의무화한 Root Cause 기록을 읽어 회귀 TC를 설계하고 시트·티켓까지 연결하는 Claude Agent. 정책을 만든 사람이 그 정책이 남긴 데이터를 쓰는 도구를 직접 만들었습니다.

Claude Agent

MCP

3단계 게이팅

안전장치

사내 확산

[자세히 보기 →](#)

05 · PRACTICE

검증 실무 — 산출물 · API 검증 · 업무 도구

SWR · TC · Defect · Xray 추적성 같은 QA 표준 산출물과, UI 뒤를 직접 두드리는 API 검증, 반복 업무를 줄이려고 만든 유틸리티들.

Xray 추적성

TC 설계 기법

API 검증

Postman

Python 도구

[자세히 보기 →](#)

CAREER & PROJECTS

경력과 프로젝트

도메인은 자동차 임베디드 → 공공·교육 웹 → 산업 AI 비전으로 바뀌었지만, 들어가서 한 일의 순서는 늘

같았습니다. 회사별로 맡은 프로젝트를 함께 정리했습니다.

2026.02 - 현재

아하랩스

QA Engineer · SW QA 담당

산업 AI 비전 검수 플랫폼 Daisy의 품질 검증 총괄. QA 프로세스 0 → 1 구축, MLOps E2E 회귀 자동화 1인 구축·운영, 다수 고객사 배포 QA 동시 운영.

프로세스와 자동화를 둘 다 0에서 설계하고, AI를 워크플로우에 편입시킨 경험.

▼ 프로젝트 6건

Daisy · 대한송유관공사 · 부천 · 기아자동차 · LG U+ · 뉴퍼마켓

Daisy AI Ops Platform

2026.02 ~ 현재 · QA Engineer

스마트팩토리 AI 비전 검사 플랫폼. 기능 검증·배포 QA와 함께 **QA 프로세스를 0에서 구축**했습니다.

- 워크플로우 · 배포 게이트 · RC 검증 · RCA 체계 설계 →
- MLOps 무인 회귀 검증 · 셀프서비스 러너 →
- Claude Agent 기반 회귀 TC 자동 생성 →

요구사항 ↔ TC ↔ 결함 추적성 체계를 구축해 신규 기능 요구사항 커버리지 **100%** 달성.

대한송유관공사 — AI 누유 검출

2026.04 ~ · 검증 담당 · 호서 구간 시범

배관 감시 시스템의 **오알람 저감**을 위한 AI 누유 예측 모델과 대시보드 검증.

- **지표를 어떻게 세는지부터 확인** — 누유감지율·오알람률의 1분 단위 집계 기준 검증. 모델이 맞았는지보다 **맞았다고 판정하는 방식**이 먼저
- 압력 **초당 1,000건** 시계열 처리와 TCP 소켓 준실시간 수집 연동
- **통신 단절·재연결 데이터 유실** 시나리오와 캐시 보존 제약 반영 장애 대응

부천 — 필터 원단 불량 검출

2026.05 ~ 2026.07 · 검증 담당 · 3개 라인

광학계와 AI PC로 불량을 실시간 검출하고
검단기를 정지시키는 솔루션 도입 검증.

- 설비가 실제로 멈추는지까지가 검증 범위
— 촬영 → 이상 감지 → 정지 트리거
전송·수신 → 검출 내역 확인 → 재구동
E2E
- AI PC 1대 : 검단기 1대 구성에서 3개
라인 동시 운영 실시간 처리
- 조명·광학계 고장 상태가 흔들릴 때의
검출 편차 확인

기아자동차 — 토크 검사 ABS

2026.06 ~ 2026.08 · 검증 담당 · PoC

토크 스패너 측정값이 차량의 어느
체결부에서 측정된 것인지를 영상
딥러닝으로 판별하는 과제의 타당성 검증.

- 목적에 맞는 판정 기준을 세웠습니다 —
완전 판별이 아니라 실현 가능성 확인이
목적인 PoC라, 통과/실패가 아닌 *가능
여부의 근거*를 산출물로
- 촬영 각도 · 조명 · 거리를 달리한 영상
세트로 판별 가능 여부 검증
- 오판 케이스 분류, 작업 순서 오류
시나리오로 기존 순서 기반 저장 방식의
한계 확인

LG U+ — KLK 온톨로지 대시보드

2026.06 ~ · 검증 담당

데이터 · 메타데이터 · 비즈니스 용어를 AI
온톨로지로 구조화해 관리·운영하는
대시보드 검증.

- 한 곳을 바꿨을 때 연결된 것이
따라오는지 — 용어 등록·수정·삭제 시
연결된 온톨로지 반영 **정합성** 검증
- 데이터 · 메타데이터 · 비즈니스 용어 간
관계 매핑 결과 검증

뉴퍼마켓 — LLM 기반 ERP 매칭

2026.02 ~ 2026.04 · QA Engineer

LLM으로 ERP 품목을 자동 매칭하는
시스템의 기능 검증과 응답 정확도 검증.

- LLM 매칭 결과의 정확도 검증 기준을
수립 — 정답이 하나로 정해지지 않는
출력을 판정 가능한 형태로
- Figma Export 기반 UI 요구사항 분석
→ TC 설계
- HTTP 통신 보안 취약점 조사 → 개선
사항 도출

소디스

기술연구소 주임연구원 · SW QA 담당

QA 조직이 부재한 환경에 애자일 스크럼 · Git 브랜치 전략 · Jenkins CI/CD 직접 수립. Playwright 자동화 최초 도입, 자율주행 · 커넥티드카 국책과제 검증 수행.

표준이 없는 곳에 프로세스와 CI/CD를 세워본 경험.

▼ 프로젝트 3건

커넥티드카 CCU · 대전 자율주행 · 공간정보진흥원 LMS

커넥티드카 CCU 국책과제

2025.08 ~ 2026.01 · QA Engineer

차량 텔레매틱스 CCU의 원격제어 · 데이터 송수신 검증과 소프트웨어 신뢰성 검사.

- **KOLAS 공인 소프트웨어 신뢰성 검사 통과**
- CANoe 기반 HILs 장비 네트워크 송수신 검증, DCM 연동 CAN 통신 검증
- Linux 기반 보안 테스트 · 사이버보안 검증 절차서/결과서 작성

대전 자율주행 상용화지구

2025.06 ~ 2026.01 · QA Engineer · ETRI 주관

자율주행 버스 애플리케이션의 기능 검증 및 보안 취약점 분석.

- **실차 검증에 직접 참여** — 현장에서 품질 이슈 대응 및 개선
- CAN 통신 · ETRI 서버 프로토콜 기반 송수신 검증
- Jira 기반 결함/테스트 이슈 관리 체계 구축·운영

공간정보진흥원 LMS

2025.06 ~ 2026.01 · QA

교육포털 학습관리시스템 개발·유지보수.
자동화를 처음 도입한 프로젝트입니다.

- **Playwright 기반 UI 자동화 도입 · Jenkins CI/CD 구축**
- **Xray 도입으로 TC-결함 추적성 확보 →**
- KRDS 웹 접근성 반영 UI 요구사항 정의서, REST API 검증 →

2023.02 - 2025.06

와이드넷 엔지니어링

미러링검증1팀 주임 · 모빌리티 검증 테스트 엔지니어

TMAP AUTO for Mercedes-Benz 내비게이션 시스템 검증. ASPICE Level 1 SWE.6 인증 획득 기여, HILS 기반 조기 검증 환경 구축.

정형화된 표준을 지키며 대규모 검증을 굴러본 경험.

▼ 프로젝트 1건

TMAP AUTO for Mercedes-Benz

TMAP AUTO for Mercedes-Benz

2023.03 ~ 2025.06 · Test Engineer

TMAP MOBILITY와 협업해 Benz 차량에 탑재되는 내비게이션 시스템 검증. **정형화된 표준** 아래에서 대규모 검증을 굴러본 경험입니다.

- **ASPICE Level 1 SWE.6 인증 획득에 기여**
- 2주 스프린트 기준 **700건 이상 TC 작성 및 검증**
- **HILS 테스트 환경 구축** — 실차 대비 조기 검증 구조 마련
- 비기능 KPI 측정 스크립트 개발로 **검증 리소스 30% 절감** →

SIDE PROJECTS

토이 프로젝트

회사 업무와 무관하게 **궁금해서 만들어 본 것들**입니다. 업무 성과로 내세울 것은 아니지만, 앞의 실무

도구들이 어디서 왔는지를 보여주는 자리라 남겨둡니다.

NASDAQ AUTO — Multi-Agent AI 시스템

TypeScript · Node.js · 개인 프로젝트

서로 다른 AI 모델 3종(GPT · Gemini · Claude)이 각자의 관점에서 분석하고, **Manager Agent가 세 보고서를 통합해 최종 결론을 내리는** Multi-Agent 협력 구조를 설계 · 구현했습니다. 스케줄러 → 데이터 수집 → 병렬 분석 → 통합 → 리포트 발송까지 22개 모듈의 자동화 파이프라인입니다.

QA 관점에서 남은 것 — 서로 다른 모델의 관점을 교차 검증해 편향을 줄이는 구조와, 자동 판단에는 반드시 근거 기록이 따라와야 한다는 원칙. 이 감각이 Regression TC Creator의 안전장치 설계로 이어졌습니다.

github.com/kx2471/NASDAQ_AUTO →

OCR 활용 매크로

Python · Pillow · pytesseract · 개인 프로젝트

OCR 기술이 궁금해서 만들어 본 도구입니다. 지정한 화면 영역을 일정 간격으로 캡처하고 그 영역에서 텍스트를 자동 추출합니다. 캡처 이미지를 로컬에 보존해 **추출 실패 시 원인을 되짚을 수 있게** 했습니다.

여기서 이어진 것 — 이때 익힌 OCR 파이프라인이 그대로 실무 도구로 넘어갔습니다. Clipboard Translate의 이미지 텍스트 추출부가 이 실험에서 출발했고, *OCR 기반 비정형 UI 테스트 자동화*라는 아이디어도 여기서 얻었습니다.

실시간 로그 화면 모니터링, UI 내 특정 텍스트 인식 등에 활용했습니다.

이 둘을 남겨두는 이유

업무 성과가 아니라 “궁금하면 만들어 본다”는 습관 자체가 실무 도구의 출처이기 때문입니다. OCR 매크로 → 클립보드 번역기, Multi-Agent 실험 → QA 워크플로우용 Agent로 이어졌습니다.

NASDAQ AUTO는 개인 학습 목적의 프로젝트이며 투자 참고용으로만 제작되었습니다. 투자자문이 아닙니다.

더 보기



GitHub

직접 만든 자동화 도구와 개인 프로젝트
저장소.

github.com/kx2471 →



포트폴리오 (PDF)

인쇄·오프라인 열람용. 이 사이트 전체를
담았습니다.

PDF 내려받기 →

연락처 — 010-4482-2033 · kx2471@gmail.com

김상옥 · Kim Sang-Ok — SW QA Engineer

본 포트폴리오의 문서와 화면은 실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사·담당자명·버전·
내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.

QA 프로세스를 0에서 1로

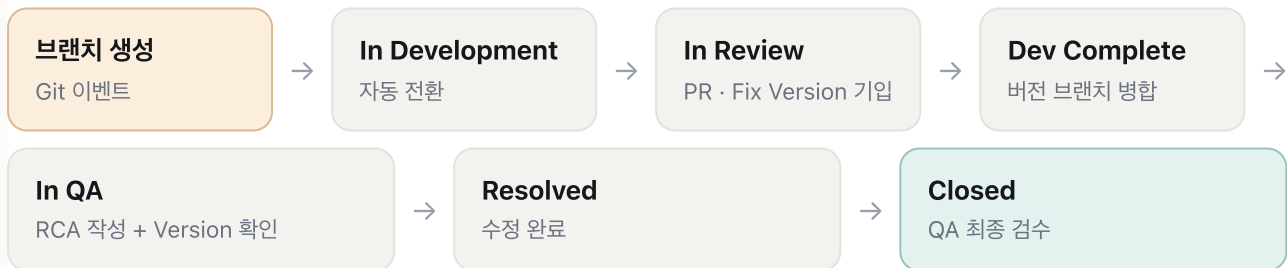
이슈 관리 체계가 없던 환경에 워크플로우 · Priority 기준 · Root Cause 의무화 · 배포 게이트 · RC 검증 절차를 직접 설계하고, 규칙이 실제로 동작하도록 자동화까지 붙였습니다.

PROBLEM 우선순위는 작성자 주관에 맡겨졌고, 버그 원인은 기록되지 않았으며, 미검증 빌드가 현장에 배포되는 사고가 있었습니다.	ACTION 판단 기준을 명문화하고, 지켜지지 않으면 자동으로 되돌리는 규칙을 트래커에 걸었습니다.
RESULT “이거 QA 봐도 되나요?”가 필드 조건으로 치환되고, 배포 차단권이 QA에 명시적으로 생겼습니다.	

▼ 개발·QA 워크플로우 — 코드의 흐름과 일의 흐름을 연결

Git x 이슈 트래커 x Automation

설계한 전체 흐름



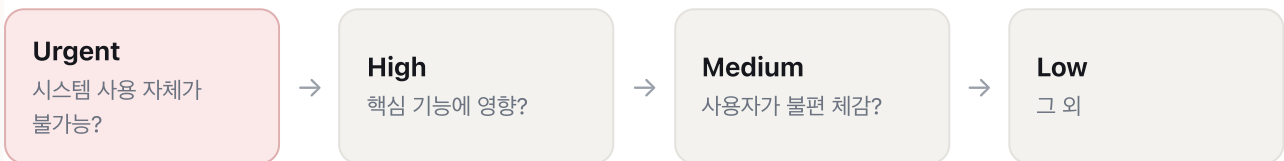
마지막 문을 QA가 닫는다 — Resolved 는 개발이 “고쳤다”고 선언한 상태일 뿐입니다. QA가 최종 검수한 뒤에만 Closed 로 넘어갑니다. 티켓을 여는 사람과 닫는 사람을 분리해, 고쳤다는 말과 고쳐졌다는 확인이 섞이지 않게 했습니다.

되돌아가는 길도 설계 — 검증 반려 시 In QA → In Development 로 복귀. RCA 미작성 상태로 Dev Complete 를 시도하면 Automation이 자동으로 In Development로 복구합니다.

바꾼 것들

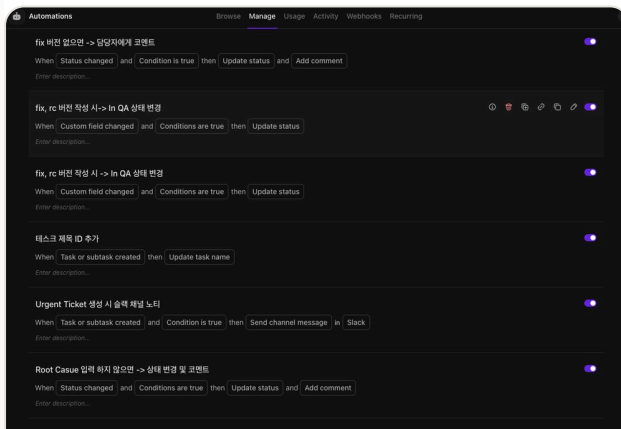
영역	기존	도입한 변화
Priority 판단	작성자 주관에 의존	4단계 판단 Flow + 핵심 기능 목록 · 등급별 처리 SLA
티켓 유형	Bug/Task 존재, 통계 불가	Task Type 분리 → Bug에만 RCA 자동화 연결
결함 기록	"수정 완료"만 남고 원인 유실	Root Cause / Corrective Action 의무 작성
상태 전환	수동 전환, 누락·역행 빈번	Git 이벤트 → 트래커 상태 자동 전환
QA 진입 조건	구두 확인 필요	Fix/RC Version 기입을 조건으로 자동화
브랜치 운영	명명 규칙 부재, 삭제 시점 모호	4종 브랜치 정의 + 네이밍·생애주기 규칙

Priority — 누가 작성해도 같은 결과가 나오도록

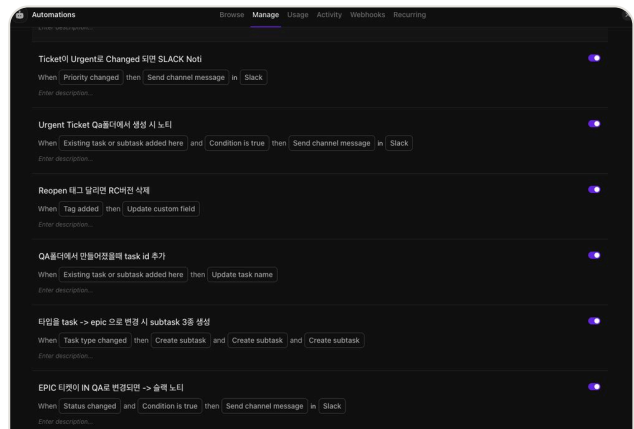


“내 티켓은 다 Urgent”가 반복되던 상황이었습니다. **핵심 기능 목록**(검사 실행·판정, 결과 조회, 모델 학습/배포, 사용자 인증, 데이터 저장·조회)을 따로 정의하고, 등급별 처리 SLA까지 약속해 Priority가 단순 라벨이 아니라 **실제 처리 흐름**으로 이어지게 했습니다.

규칙은 실제로 걸려 있습니다



상태 · 커스텀 필드 변경을 트리거로 상태 전환과 코멘트를 자동 실행한다. 맨 아래가 Root Cause 미입력 시 상태를 되돌리는 규칙.



우선순위 · 태그 · 타입 변경까지 트리거로 활용한다. Urgent 승격 즉시 Slack 공지, Epic 전환 시 하위 작업 자동 생성.

앞의 표와 대조해 보면

- Root Cause 입력 하지 않으면 → 상태 변경 및 코멘트 — RCA 미작성 시 되돌리는 규칙
- fix, rc 버전 작성 시 → In QA 상태 변경 — 버전 필드를 QA 진입 조건으로
- Ticket이 Urgent로 변경되면 SLACK Noti — Priority SLA의 “즉시 전체 공유”를 규칙으로 보장

화면은 운영 중인 규칙 목록의 일부입니다. Epic 전환 시 하위 작업 3종 자동 생성, Reopen 태그 시 RC 버전 초기화, 풀더 기준 티켓 ID 자동 부여 등 **티켓 생애주기 전반에 걸쳐** 손이 가던 지점들을 규칙으로 옮겼습니다.

QA Request 봇 — 요청 접수를 시스템으로

Slack · AWS Lambda · ClickUp API

여러 프로젝트의 QA 요청이 DM·구두·여러 채널로 흩어져 들어왔고, 버전·일정·범위가 빠진 채 오기 일쑤였습니다. 슬래시 명령 하나로 접수를 일원화하고, 이후 알림까지 자동화했습니다.

/qa-request

Slack Slash Command



AWS Lambda

모달 호출 → 검증 → API



티켓 생성 + 채널 공지

이후 알림이 같은 스레드에 축적



QA 티켓 생성



티켓 제목

ex) Daisy Pro 1.9.0 QA 요청

프로덕트 (프로젝트)

프로덕트 선택

테스트 목적

ex) 1.9.0 배포를 위한 QA 요청

테스트 버전 (옵션)

ex) 1.9.0

💡 버전은 숫자.숫자.숫자 형식으로 입력해주세요. ex) 1.9.0

변경 범위

ex)
데이터셋 비디오 기능 추가
모델 개발 파이프라인 기능 추가

취소

생성

슬랙에서 /qa-request 입력 시 뜨는 요청 양식. 제목 · 프로덕트 · 목적 · 버전 · 변경 범위 · 일정 · 배포 예정일 · 고객사 배포일을 받는다.

테스트 버전 (옵션)

a.b.c

● 버전 형식이 올바르지 않습니다. 숫자.숫자.숫자 형식으로 입력해주세요. ex) 1.9.0

형식이 맞지 않으면 실시간 경고가 뜨고 제출이 막힌다.

테스트 버전 배포 예정일

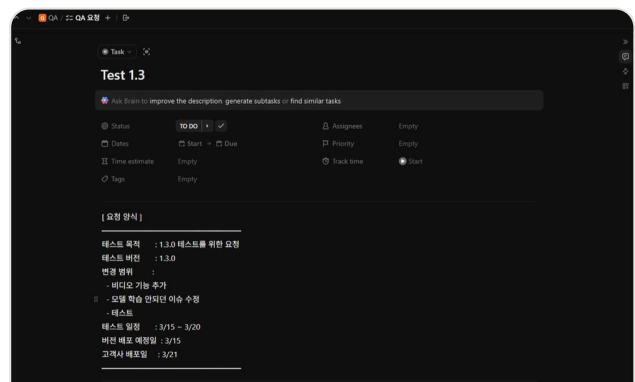
날짜 선택

시간

시간대: 서울

🕒 이 시각까지 QA 버전(RC1)이 배포되지 않으면 리마인드 알림이 발송됩니다.

배포 예정 시각. 이 값이 리마인더의 기준점이 된다는 설명을 입력 칸 바로 아래에 붙였다.



제출 즉시 생성된 ClickUp 티켓. 본문이 [요청 양식] 블록으로 정리되고 날짜·커스텀 필드가 자동 입력된다.

발송 후 — 약속이 지켜지는지까지 감시

QA Ticket Bot 앱 오늘 오후 4:47
 17 Start Date / Due Date 변경 @김상욱(마케팅/기획 - QA) @
 - 티켓: test 9.9.9
 - 변경자: 김상욱
 - 2026-07-09 (목) ~ 2026-07-11 (토) → 2026-07-14 (화) ~ 2026-07-17 (금)

일정 변경 시 관련자 멘션과 함께 변경 전 → 후가 스레드에 자동 공지된다.

QA Ticket Bot 앱 오늘 오후 4:31
 ✓ QA 티켓이 생성됐습니다!
 요청자:
 제목: 검증 버전 배포 성공
 프로덕트: Daisy Pro
 테스트 일정: 7/9 (목) ~ 7/10 (금)
 ClickUp 링크: <https://app.clickup.com/>

1개의 댓글

QA Ticket Bot 앱 오늘 오후 4:31
 ✓ QA 버전 배포 확인됨
 - 티켓: 검증 버전 배포 성공
 - Daisy Pro 1.7.0 → 1.7.0-RC1 빌드 성공 확인 (예정 시각 전 배포)

예정 시각 전에 빌드가 확인되면 “배포 확인됨”으로 스스로 닫는다.

QA Ticket Bot 앱 오늘 오후 4:23
 ✓ QA 티켓이 생성됐습니다!
 요청자:
 제목: QA 버전 배포 성공 케이스
 프로덕트: Daisy Pro
 테스트 일정: 7/9 (목) ~ 7/10 (금)
 ClickUp 링크: <https://app.clickup.com/>

2개의 댓글

채널에도 전송됨

QA Ticket Bot 앱 오늘 오후 4:25
 ⚠ QA 버전이 배포되지 않았습니다! @김상욱(마케팅/기획 - QA)
 - 티켓: QA 버전 배포 성공 케이스
 - 프로덕트: Daisy Pro / 버전: 1.7.0
 - 배포 예정: 7/8 (수) 16:25
 - release 채널에서 DAISY-PRO-RC 1.7.0-RC1 빌드 성공을 찾지 못했습니다. (배포 확인까지 2시간 간격으로 다시 알립니다)

예정 시각까지 빌드를 찾지 못하면 경고 + 관련자 멘션. 배포될 때까지 2시간 간격으로 재알림.

티켓을 만드는 데서 끝내지 않았습니다

봇이 **release** 채널의 **Jenkins 빌드 메시지를 감시**하면서, 티켓에 적힌 버전의 RC1 빌드가 올라왔는지 스스로 확인합니다. 예정 시각이 지나도 못 찾으면 경고 + 관련자 멘션, **2시간 간격 재알림**, **24시간 후 자동 종료**.

상황	순진하게 만들면	실제 설계
날짜 최초 설정	변경으로 보고 알림	알림 없음 — 변경·삭제일 때만
시작일·마감일 동시 변경	알림 2건	1건으로 병합
예정 시각 전 배포	예정 시각까지 대기	확인 즉시 ✓ 처리
미배포 지속	1회 경고 후 방치	2시간 간격 재알림
계속 안 나옴	무한 반복 → 알람 피로	24시간 후 자동 종료
배포일이 변경됨	옛 시각 기준 오탐	1분 내 감지 → 새 기준 재동작
티켓이 삭제됨	고아 리마인더가 계속 올림	1분 내 자동 정리
판정 규칙 없는 제품	오탐 알림	티켓만 생성 · 리마인더 미동작

여기서 배운 것

알림 시스템의 진짜 난이도는 “언제 안 보낼지”에 있습니다. 자동 알림은 만들기는 쉽고 미움받기도 쉽습니다. 팀이 붓을 끄지 않게 만드는 것이 설계의 목표였습니다.

이 작업이 보여주는 것 — 규칙을 만드는 사람에서 나아가 규칙이 동작하도록 시스템을 직접 빌드했고, Slack · 서버리스 · 트래커 API · CI 알림이라는 **이질적 시스템을 하나의 경험으로 연결했습니다.**

▼ 배포 게이트 — QA의 NO-GO 차단권

인시던트에서 출발한 프로세스

미검증 빌드가 고객사 현장에 배포된 인시던트가 있었습니다. 원인을 추적해보니 개인의 실수가 아니라 **구조의 공백**이었습니다 — 배포 대상 확인이 버전 라벨 기준으로만 이뤄졌고, QA 검증 완료와 배포 결정 사이에 **명시적 연결 고리가 없었습니다.**



권한을 나눴습니다 — QA는 품질 상태를 보고하고 **NO-GO로 배포를 차단**할 수 있고, **go/no-go 최종 결정**은 **PM**에게 둡니다. 품질 보고 책임과 배포 결정 권한이 섞이지 않게 한 설계입니다.

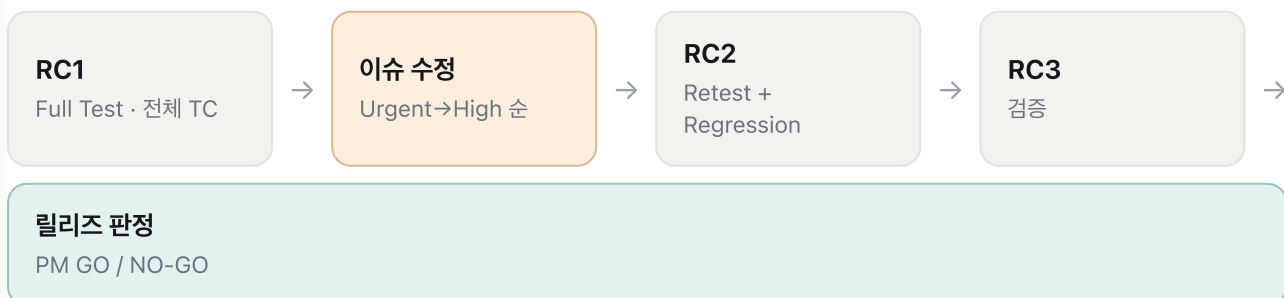
예외를 프로세스 안으로 — 긴급 배포를 부정하는 프로세스는 우회됩니다. 비상 레인을 따로 두되 **위험 감수 승인 · 미검증 범위 문서화 · 사후 QA 의무화**를 붙였습니다.

인시던트를 **책임 추궁이 아닌 프로세스 개선**으로 전환했고, QA가 릴리즈 파이프라인의 형식적 참여자가 아니라 **명시적 권한을 가진 게이트키퍼**가 됐습니다.

▼ RC 검증 절차 — “언제 끝나는가”를 숫자로

Exit Criteria · 역할 분리

RC 검증이 “상황마다 다르게” 진행되던 것을 **반복 가능한 절차**로 정립했습니다. QA 검증과 개발 수정이 병행되고, 준비되는 즉시 배포(상한 ≤1영업일)해 조기 수렴시킵니다.



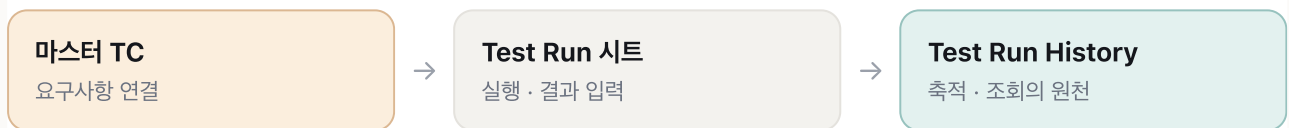
Exit Criteria	기준
잔여 Urgent	0건 (우선 게이트)
잔여 High	5건 미만 (건별 PM·PL 협의)
Pass Rate	95% 이상
Normal / Low	개별 게이트 없이 Pass Rate에 포함 → Known Issue로 관리

개별 건을 놓고 매번 논쟁하는 대신 **숫자로 판정 가능한 기준**을 정의해 판정 비용을 줄였습니다. **QA는 충족 여부를 보고하고, GO/NO-GO 결정은 PM이 합니다** — QA가 비즈니스 판단을 떠안지 않고 품질 사실에 집중할 수 있게 한 분리입니다. 프로덕션 긴급 수정은 Full Test 없이 자동화 기반 빠른 검증만 거치는 **Hotfix 별도 트랙**으로 분리해, 긴급 수정이 정규 절차를 무너뜨리지 않게 했습니다.

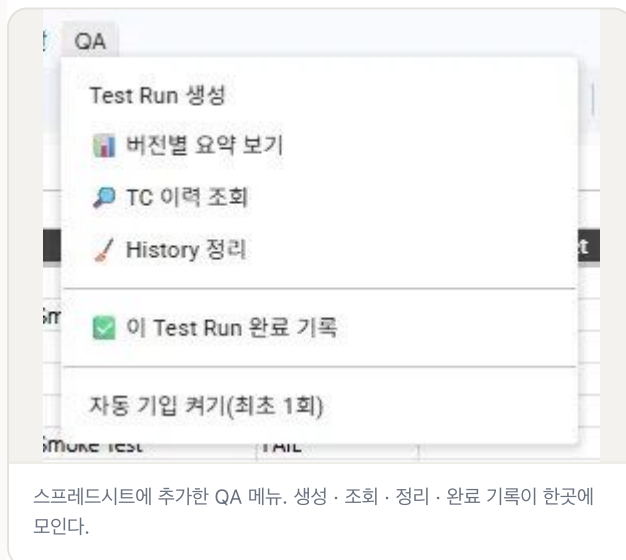
▼ QA Test Run 관리 도구 — 결과를 하나의 이력으로

Google Sheets + Apps Script

버전마다 마스터 시트에서 TC를 복사하고 결과를 각 버전 시트에 흩어 기록하다 보니, **“이 TC가 버전을 거치며 어떻게 됐나”**는 여러 시트를 눈으로 따라가야 했습니다. 회귀 대상 선별 · 버전별 요약 · 추적성 근거도 매번 수동이었습니다.



쌓는 곳과 보는 곳의 분리 — History는 결과를 세로로 계속 쌓는 원천이고, 사람은 그 위에 얹은 조회 뷰로 봅니다. DB 원본 테이블을 직접 뒤지지 않고 쿼리로 조회하는 것과 같은 구조입니다.



TC 이력 조회

TC 이력 · TC_01

표 복사

닫기

버전	TC_ID	기능영역	우선순위
TestRun_20260721 (2)	TC_01	ADMIN	
TestRun_20260721	TC_01	ADMIN	

TC 이력 조회 — 하나의 TC가 버전을 거치며 어떻게 됐는지 세로로 쌓인 History에서 뽑아 보여준다.

QA Test Run 관리 도구 사용 가이드

Google Sheets 기반 Test Run 생성 · 실행 · 이력 관리 자동화

Aha Labs · QA · Dailly Platform | 작성: 김상욱

1. 개요

이 도구가 해결하는 문제

기존에는 버전(RC)마다 마스터 시트에서 TC를 통째로 복사한 뒤 필요 없는 항목을 수동으로 지우고, 실행 결과를 각 버전 시트에 붙여서 기록했습니다. 이 방식은 **버전 → TC 방향만** 보였고("이번 버전에 뭐가 있나"), 정작 **TC → 버전 방향**("이 TC가 버전을 거치며 어떻게 됐나")은 여러 시트를 눈으로 따라가야 했습니다. 회귀 대상 선택·버전별 요약·추적성 근거도 매번 수동이었습니다.

이 도구가 만드는 흐름

생성 → 실행 → 완료 기록의 3단계를 자동화하고, 모든 결과를 하나의 이력(History)에 축적합니다. 추적성의 핵심인 "요구사항 → TC → 실행 결과 → 결함"에서 **결과 축**을 실패데이터로 확보합니다.



핵심 원칙 — 쌓는 곳과 보는 곳의 분리

History는 결과를 **세로로 계속 쌓는 원천(원본)**이고, 사람은 그 위에 얹은 **조회 뷰(버전별 요약 · TC 이력)**로 봅니다. 데이터베이스 원본 테이블을 직접 뒤지지 않고 쿼리로 조회하는 것과 같은 구조입니다.

2. 본문

2-1. QA 메뉴 구성

메뉴	기능
Test Run 생성	마스터/기존 시트에서 조건 필터로 새 Test Run 시트 생성
버전별 요약 보기	버전별 총 TC / PASS / FAIL / NA / Pass Rate 팝업
TC 이력 조회	특정 TC의 버전별 결과 변천사 팝업
History 정리	삭제되어 번 종료 남은 이력 행 정리
이 Test Run 완료 기록	현재 시트 결과를 History에 기록(확정)

도구와 함께 작성한 사용자 가이드, 개요 · 메뉴 구성 · 최초 설정 · 생성/실행/완료/조회 · 유의사항으로 구성.

회귀 승계 — RC2를 만드는 법

소스를 기존 **Test Run 시트**에서 RC1로 두고 필터를 **P/F = FAIL, NA** 로 걸면, **RC1에서 깨졌거나 미검중인 항목만** P/F 빈 채로 승계됩니다. RC 검증 절차의 "Retest + Regression"을 손이 아니라 도구가 계산합니다.

이 도구로 답할 수 있게 된 질문

질문	답하는 방법
커버리지 — 이번 버전에 뭘 검증했나?	Test Run 시트 + 통계 블록
회귀 대상 — 과거 FAIL 난 TC는?	History 필터 / 승계 생성
배포 판단 — 이 버전 Pass율은?	버전별 요약 (Exit Criteria와 직결)
추적성 — 이 TC가 버전마다 어땠나?	TC 이력 조회

확산까지 설계

자동 기입 트리거는 **스프레드시트 자체에 설치**되어 누가 편집하면 동작합니다. 팀원이 각자 켤 필요가 없습니다. 도구를 나눠주는 것과 팀이 실제로 쓰게 만드는 것은 다른 문제입니다.

근본원인 분석 — 원인이 쌓이는 구조

전수 리뷰 · 10개 카테고리

버그 티켓은 쌓이는데 근본 원인은 자유 서술로 흩어지거나 아예 기록되지 않았습니다. "어떤 유형의 원인이 반복되는가"를 아무도 정량적으로 답할 수 없었고, 구조화된 필드를 도입하자고 제안해도 **설득 근거가 없었습니다**.

자유 서술 · 미기록
집계 불가



전수 분석 (1회)
귀납적으로 10개
카테고리 도출



선택형 필드 + 작성
의무화
Automation이 강제



원인 분포 자동 축적
이후 모든 버그가
데이터가 된다

"몇 건을 분석했는가"는 중요하지 않습니다. 분석 시점의 건수는 계속 늘어나고, 그것이 이 체계가 의도한 결과입니다. 목적은 한 번의 정확한 집계가 아니라 **버그가 늘어나는 동안에도 원인 분포가 자동으로 최신 상태를 유지하는 것이었습니다.**

표본이 아니라 전수를 봤습니다. 분류 체계를 먼저 정의하고 티켓을 끼워 맞추는 것이 아니라, 실제 티켓에서 반복 패턴을 추출해 카테고리를 만들었기 때문에 현장의 결함 양상을 그대로 반영합니다.

이 데이터가 흘러가는 곳

- 품질현황보고의 **정량 근거** — 반복 원인 상위 카테고리 기준으로 개선 우선순위 도출
- 정기 트리아지 · Known Issue 백로그 관리의 기초 데이터
- **Regression TC Creator의 입력** — Agent가 이 필드를 읽어 회귀 TC를 설계 → 04 AI Agent

핵심 — "제안하기 전에 전수를 본다"는 접근. 필드 하나를 추가하는 작은 프로세스 변경도 데이터 근거 위에서 제안하면 조직의 수용 속도가 달라집니다.

김상옥 · Kim Sang-Ok — SW QA Engineer · kx2471@gmail.com · github.com/kx2471

실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사 · 담당자명 · 버전 · 내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.

MLOps 파이프라인 무인 회귀 검증

데이터셋 업로드부터 7종 AI 모델 학습·배포까지, 비전 AI 학습 플랫폼의 전체 파이프라인을 Playwright로 자동 검증하는 회귀 시스템을 **1인으로 설계·구축·운영**하고 있습니다.

PROBLEM

릴리스마다 수동으로 하루 이상 걸리던 회귀 검증.
실서버 · 단일 공유 계정 · 30분~3시간짜리
학습이라는 제약까지 걸려 있었습니다.

ACTION

제약을 먼저 정의하고, 테스트가 서로를 깨뜨리지
않는 구조를 설계했습니다.

RESULT

명령 한 줄 무인 실행 + Slack 자동 보고. 스위트가
2배 이상 늘어나는 동안 아키텍처는 다시 쓰지
않았습니다.

▼ 먼저 제약을 정의했다 — 아키텍처의 모든 결정이 여기서 나왔습니다

5가지 제약

테스트를 짜기 전에 이 제품이 자동화에 무엇을 허용하지 않는지부터 정리했습니다.

제약	그래서 무엇이 어려워지는가	설계로 답한 방식
산출물의 소속 모든 노드는 자기 프로젝트 안에서만 유효	고정 프로젝트를 재사용하면 실행할 때마다 노드가 쌓여 결국 깨진다	테스트마다 전용 프로젝트 생성 → 종료 시 삭제를 공용 헬퍼로 강제
실서버 대상 목(mock) 없이 3개 실행경	실패를 임의로 만들 수 없고, 서버 상태가 결과에 개입한다	서버 정책을 실험으로 규명해 전제로 삼고, 장애는 요청 차단·주입으로 재현
단일 공유 계정 + SSO	동시 로그인인 앞 세션을 끊어 병렬 실행이 성립하지 않는다	세션 단위가 계정이 아니라 기기(User-Agent) 임을 규명 → worker별 UA 분리
고비용 작업 모델 학습 30분~3시간	소비 테스트마다 재학습하면 스위트를 운영할 수 없다	학습 결과를 픽스처에 기록해 재사용 — 타입당 1회로 고정
비동기 장기 작업 완료 신호가 일시적 UI	토스트를 기준으로 삼으면 부하 시 false PASS가 난다	노드 상태 · 다운로드 파일 · 요청 페이로드를 판정 근거로

화면이 아니라 데이터를 단언한다 — 리포트 검증은 화면 표시만 보지 않고 다운로드한 zip을 파싱해 서버 원본
값과 대조합니다. 이미지 수 · 모델 사이즈 · Threshold가 UI · 파일 · 요청 페이로드 세 곳에서 일치하는지
확인합니다. “떠 보이는 것”과 “실제로 된 것”을 구분하는 것이 판정 기준입니다.

▼ 병렬 실행 — 단일 계정 실서버에서 성립시키기

핵심 난제

🔑 worker별 User-Agent 분리

SSO가 "같은 UA 동시 로그인 = 같은 기기 재로그인"으로 앞 세션을 끊는 것을 **실험으로 규명**하고, worker마다 UA 토큰을 부여해 **3-worker 동시 로그인** 성립.

⚡ 파일 내 병렬

단일 직렬 전처리 스위트를 3그룹으로 분할, 각 그룹이 전용 프로젝트를 갖고 worker에 분산.

19.3분 → 11.8분 (-39%)

🏢 서버 정합

서버 **동시 작업 상한(3)**에 맞춰 worker 수와 대기 정책을 정합, 완료 대기 한도를 "최장 학습 잔여 + 여유"로 산정해 큐 대기 오염 제거.

버린 것도 기록합니다

세션 문제 우회용으로 만들었던 탭 orchestration 구조는 **단일 실패점**이므로 폐기했습니다. 더 단순한 정답이 나오면 이미 만든 것도 버립니다.

▼ 트러블슈팅 — 증거로 원인을 확정한 5건

추측으로 고친 것은 없습니다

CASE 1 — 로그인은 됐는데 로그인 화면으로 돌아온다

로그인 직후 goto 로 끼어들면 SSO 리다이렉트 체인이 끊겨 미인증이 됨을 단계별 URL 진단으로 확정. 이동을 금지하고 자동 도달만 대기하도록 변경, 신/구 로그인 방식을 자동 분기해 **3개 환경이 한 헬퍼를 공유**.

CASE 2 — "수동은 되는데 왜 자동화만 안 되지?"

"계정당 단일 세션"이라는 그럴듯한 가설에 머물지 않고 **모순을 실험으로 파고들어**, 세션이 계정이 아니라 기기 단위임을 규명. cloud 병렬 **1 → 3 worker** 복구.

CASE 3 — 6분짜리 학습이 26초 만에 "완료"

통과한 TC가 범인인 케이스. 리포트 타임라인 재구성 + DOM 스냅샷으로 준비 구간이 부하에서 길어져 완료 판정을 오판함을 확인. 진행 토스트를 판정에 추가해 전 구간 보호.

CASE 4 — 캔버스가 하얗게 비는 글리치

trace DOM 스냅샷 추적으로 React Flow의 인라인 `visibility:hidden` 고착을 확인, 트리거(모달 닫힘 직후 소켓 알림)까지 특정.

단독 재현 스크립트를 제작해 제품 버그로 리포트.

CASE 5 — 40초가 모자라서 실패한 검증

"재현 안 되는 flaky"를 전체 테스트의 시작/종료 시각으로 서버 슬롯 점유를 재구성해 규명 — 코드 버그가 아니라 구조적 대기. flaky를 "운"이 아니라 원인 있는 결함으로 다뤘습니다.

🔍 공통 접근

- trace DOM 스냅샷 시점별 추적
- 리포트 타임라인 재구성
- 요청 페이로드 가로채기
- 단독 재현 스크립트 제작
- 가설과 모순되는 사실을 끝까지 파고들기

▼ 리포팅과 성과 — 결과가 팀에 도달해야 자동화가 완성된다

커스텀 Slack 리포터 자작



실행이 끝나면 리포터가 결과 카드를 채널에 올린다. PASS/FAIL/FLAKY/SKIP 네 상태와 소요 시간, 검증한 스위트명이 함께 남는다.

- **경량 첨부 설계** — video·trace 제외 zip만 첨부(수 GB → 1~2MB), 로컬에는 전체 아티팩트를 실행별 폴더로 보존
- **대화형 실행기** — 명령 한 줄로 테스트명·Slack 전송 여부를 묻고 실행. CI에서는 무질문 자동, 부분 실행이 팀 채널을 오염시키지 않도록 전송은 opt-in
- **보안** — trace에 평균 기록되는 자격 증명을 인지하고 계정 정책·gitignore·첨부 제외로 노출 경로 차단

성과

항목	Before	After
릴리스 회귀 검증	수동 · 1일 이상	명령 한 줄 · 무인 실행 + Slack 자동 보고
cloud 병렬 실행	SSO 개편 후 1 worker	3 worker 복구
전처리 스위트	직렬 19.3분	그룹 병렬 11.8분 (-39%)
모델 학습 비용	소비 테스트마다 재학습	타입당 1회 · 픽스처 재사용
스위트 확장	추가할수록 구조 수정 필요	규모 2배 이상 확장 · 아키텍처 무변경
검증의 질문	기능이 도는가	+ 결과 품질이 유지되는가 · 고친 것이 유지되는가

확장 비용을 낮추는 설계 — 초기 구축 이후 **모델 성능 비교** · **회귀 테스트** 영역이 더해지며 스위트 규모는 두 배 이상이 되었습니다. 그때 한 일은 구조를 고치는 것이 아니라 **기존 구조 위에 스위트를 얹는 것이었습니다**. 자동화의 진짜 비용은 처음 만들 때가 아니라 **늘려갈 때** 발생하기 때문에 그 지점을 먼저 설계했습니다.

이 프로젝트에서 보여준 방식

- 증거로 원인을 **확정한다** — trace DOM 스냅샷, 리포트 타임라인 재구성, 요청 페이로드 가로채기
- 가설은 **실험으로 깬다** — 그럴듯한 결론에 안주하지 않고 모순을 파고들어 진짜 정책을 찾는다
- 더 단순한 답이 나오면 **버린다** — 동작하던 구조도 단일 실패점이면 미련 없이 폐기
- 실패를 **설계한다** — 제품 버그는 실패하는 테스트로 고정해 추적하고, 의존 그래프로 실패 전파를 차단

김상옥 · Kim Sang-Ok — SW QA Engineer · kx2471@gmail.com · github.com/kx2471

실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사 · 담당자명 · 버전 · 내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.

테스트 러너 — 모두가 검증할 수 있는 환경

전사가 같은 제품을 만들고 있는데, 그 제품을 검증할 수 있는 사람은 몇 명뿐이었습니다. 만드는 사람이라면 누구나 자기가 바꾼 것을 직접 확인할 수 있어야 한다고 봤습니다. 그래서 설치도 명령어도 없이 브라우저만 있으면 검증이 돌아가는 환경을 만들었고, 설계 · 구현 · 사용자 문서화까지 단독으로 진행했습니다.

PROBLEM

같은 제품을 함께 만드는데 **품질 확인은 QA를 거쳐야만** 이뤄졌습니다. 스위트는 완성됐지만 터미널·환경 설정을 아는 사람만 돌릴 수 있었습니다.

ACTION

“도구를 모르는 사람이 눌러도 잘못될 수 없다”를 기준으로, 안전한 기본값과 자동 의존성 해결을 갖춘 검증 환경을 만들었습니다.

RESULT

만든 사람이 자기가 바꾼 것을 직접 검증하고, QA는 무엇을 어떻게 검증할지 설계하는 데 집중합니다.

품질은 QA만의 일이 아닙니다 — 전사가 하나의 제품을 만들고 있다면 만드는 사람 누구나 자기가 바꾼 것을 확인할 수 있어야 하고, QA가 할 일은 그 확인을 가능하게 만드는 것이라고 봤습니다. 검증을 QA가 독점하면 품질은 QA의 처리량만큼만 좋아지지만, 검증할 수 있는 사람이 늘면 **품질이 팀 전체의 일**이 됩니다. 이 러너는 그 전환을 위해 만들었습니다.

▼ 설계 판단 6가지 — 화면에 드러난 결정과 그 이유

실제 운영 중인 화면

1. 막되, 막는 이유를 함께 준다

3. 실행 옵션

☒ 스크린샷 하이라이트 ☐ 창 띄우기 ☐ Slack 리포트 전송 누가 이름

❗ 지금 다른 실행이 진행 중입니다 — AD 프로라벨 검증 (김상욱 · 이 PC · 16분째 실행 중) 끝난 뒤에 다시 시도해 주세요.

선택 0건

전체 풀런

예상 소요 시간 —

실행

실행이 막힐 때, 막는 데서 끝내지 않고 누가·무엇을·몇 분째 돌리는지 함께 노출한다.

공유 계정 제약은 없앨 수 없으므로 동시 실행을 차단하되, 차단이 “고장”으로 보이지 않도록 **누가 · 무엇을 · 몇 분째** 돌리는지를 같은 화면에 노출했습니다.

왜 이렇게

사용자가 기다릴지 물어볼지를 스스로 판단할 수 있으면 차단은 장애가 아니라 정보가 됩니다. 도구에 대한 신뢰는 되는 기능이 아니라 안 될 때의 설명에서 갈린다고 봤습니다.

2. 선행 의존은 사용자가 아니라 러너가 안다

2. 테스트케이스 선택

전체 카테고리

테스트케이스 이름 또는 ID 로 검색

모델 학습 (51)

☐	model-SEGM/ML-19	SEG-multi 모델 설정 (학습모델/체크포인트/클래스)	1분 미만
☐	model-SEGM/ML-20	SEG-multi 모델학습 실행 및 완료	약 5분
☐	model-SEGM/ML-21	SEG-multi 결과 리포트 확인	1분 미만
☒	model-AD/ML-01	AD 라벨링 불러오기	1분 미만
☒	model-AD/ML-02	데이터셋 날짜 설정	1분 미만
☒	model-AD/ML-03	라벨링 완료	1분 미만
☒	model-AD/ML-04	모델학습 노드 생성	1분 미만
☒	model-AD/ML-05	모델학습 노드 이름 변경	1분 미만

전체 선택

전체 해제

프로라벨 1건을 체크하면 16건이 선택된다. 15건은 러너가 자동으로 채운 선행 테스트다.

프로라벨을 검증하려면 모델이 먼저 학습돼 있어야 합니다. 이 의존 관계를 사용자가 알아야 한다면 셀프서비스가 아니므로, 러너가 선행 테스트를 자동으로 선택하고  로 표시합니다. 마우스를 올리면 무엇 때문에 포함됐는지까지 나옵니다. **1건을 체크하면 16건이 선택됩니다.**

왜 이렇게

자동으로 채우기만 하면 “왜 16건이지?”라는 불신이 생깁니다. **자동화하되 근거를 열어두는 쪽을 택했습니다.**

3. 예상 시간은 실제 실행 모델을 반영한다

1. 환경 설정

환경

언어

CLOUD

OPS

국문

영문

환경과 언어. 선택을 바꾸면 아래 테스트케이스 목록도 함께 갱신된다.



Daisy Auto Test Result

오늘 오전 9:52



서버테스트 0828

- 요청: 김상욱
- 환경 / 언어: CLOUD / 국문
- 테스트케이스 502건 · 예상 약 1시간 33분
- [진행 상황 보기](#)

1개의 댓글



Daisy Auto Test Result

오늘 오전 11:17



서버테스트 0828 — 완료

- 완료 502건
- 소요 시간: 85분 34초
- [결과 리포트 열기](#)

실행을 걸면 요청자·환경·규모·예상 시간이 채널에 공지되고, 끝나면 같은 스크린에 실제 소요 시간이 달린다.

선택 건수와 예상 소요 시간을 즉시 계산하되, 단순 합산이 아니라 **3-worker 병렬**이라는 실제 실행 방식을 반영한 값을 씁니다. 환경과 언어를 바꾸면 대상 테스트케이스 목록 자체가 함께 갱신됩니다.

왜 이렇게

어긋난 예상치는 한 번만 틀려도 다음부터 아무도 보지 않습니다. 보여줄 수 없으면 안 보여주는 게 낫고, 보여줄 거면 맞아야 합니다.

위 실행에서 **예상 1시간 33분 → 실제 1시간 25분 34초**.

4. 환경이 못 하는 일은 비활성화가 아니라 감춥다

3. 실행 옵션

☒ 스크린샷 하이라이트

☐ 창 띄우기

☐ Slack 리포트 전송

누가

이름

선택 16건 (자동 포함 15건)

전체 풀런

예상 소요 시간

약 7분

실행

옵션은 세 개뿐. 기본값 그대로 실행해도 문제가 없도록 설계했다.

옵션은 세 개로 줄이고 **기본값 그대로 실행해도 문제가 없도록** 정했습니다. 화면이 없는 서버에서는 브라우저 창을 띄울 수 없으므로 해당 옵션을 회색 처리하지 않고 **아예 감춥니다**.

왜 이렇게

고를 수 없는 선택지를 보여주는 것 자체가 혼란입니다. Slack 전송을 opt-in으로 둔 것도 같은 이유로, **부분 실행이 팀 채널을 오염시키지 않게** 하기 위해서입니다.

[illegible]

진행률은 “**끝난 수 / 전체**”와 **현재 테스트**를 함께 보여주고, 실행 로그는 실시간 스트리밍됩니다. 창을 닫았다 다시 열어도, 다른 사람이 시작한 실행에 접속해도 로그와 진행률이 그대로 복원됩니다.

왜 이렇게

학습이 포함되면 실행이 수십 분을 넘깁니다. 그동안 브라우저를 켜 두라고 요구하는 순간 사람들은 다시 안 씁니다. 중단하더라도 이미 끝난 테스트의 결과는 보존됩니다.

1

```
4. 실행 로그

| [project] 목록 정렬을 생성일순으로 전환 (수정일순에서는 새 프로젝트가 20개 밖으로 밀린다)
| [project] 카드 확보 (2회차 시도)
[15:42:56] [PROJECT-TC2] 프로젝트 이름 변경 — Rename Test — ✓ 완료
✓ 프로젝트 이름 변경: 6a8e8ae9aeece01920a01019 → "Rename Test"

[PROJECT-TC3] 프로젝트 삭제
[15:42:56] [PROJECT-TC3] 프로젝트 삭제 — ● 실행중
[15:43:00] [PROJECT-TC3] 프로젝트 삭제 — ✓ 완료
✓ 프로젝트 삭제: 6a8e8ae9aeece01920a01019
| [globalTeardown] 삭제할 픽스처 없음 (env 지정만 사용했거나 학습 안 됨)
실행이 끝났습니다 (passed).
—— 느린 테스트 TOP 10 ——
8.7s [PROJECT-TC2] 프로젝트 이름 변경
3.8s [PROJECT-TC3] 프로젝트 삭제
0.9s [PROJECT-TC1] 프로젝트 생성
—— 파일별 소요 (테스트 시간 합계, 최적화 대상 식별) ——
13.3s 3개 projectTest.js
테스트 함께 13.3s / 복시계 16.6s → 병렬-오버헤드 차이 3.3s
✓ [slack-reporter] 후송용 데이터 기록: test-results/slack-payload.json (+ slack-report.zip)
| [slack-reporter] PW_SLACK=0 — 직접 전송 skip (dev 에서 slack-report.js 로 후송 가능)
To open last HTML report run:
~[36m~[39m
~[36m npx playwright show-report resultdataW20260826_154244~[39m
~[36m~[39m
실행이 끝났습니다 — 완료 3건
```

로그 지우기

☒ 자동 스크롤

결과 리포트 열기

스크린샷 내려받기 (4장 · 4.3MB)

결과 회수 버튼은 실행이 끝나야 나타난다. 서버에서 돌아도 내려받기는 로컬로 떨어진다.

실행이 끝나면 **결과 리포트 열기 · 스크린샷 zip 내려받기** 두 버튼이 나타납니다. 서버에서 실행해도 내려받기는 로컬로 떨어지고, 새로고침하거나 나중에 다시 열어도 직전 실행 결과는 계속 회수할 수 있습니다.

왜 이렇게

결과가 실행한 사람 손에 남지 않으면 **“돌렸다”로 끝나고** 공유·후속 조치로 이어지지 않습니다. 버튼을 실행 중에는 숨긴 것도 **미완성 리포트**를 배포하는 사고를 막기 위해서입니다.

▼ 리포트 설계 — 통과한 것도 “무엇을 봤는지”를 남긴다

flaky를 별도 상태로

자동화의 결과물이 **PASS/FAIL** 숫자로만 끝나면, 사람은 그 숫자를 검증할 방법이 없습니다.



상태	테스트케이스 ID	이름	카테고리	소요
✓ 완료	common/PROJECT-TC1	프로젝트 생성	프로젝트	3초
✓ 완료	common/PROJECT-TC2	프로젝트 이름 변경 * 검증 포인트 2개 1. ✓ TC1에서 생성된 projectid 가 동일합니다. 확인 2. ✓ 변경된 프로젝트명 반영. 확인	프로젝트	9초
✓ 완료	common/PROJECT-TC3	프로젝트 삭제 * 검증 포인트 2개 1. ✓ TC1에서 생성된 projectid 가 동일합니다. 확인 2. ✓ 프로젝트 카드 삭제됨. 확인	프로젝트	4초

검증 포인트를 펼친 모습. 줄 끝의 “화면”을 누르면 그 시점 스크린샷이 열린다.

각 테스트의 **검증 포인트**를 펼치면 무엇을 어떤 순서로 확인했는지가 나오고, 줄 끝의 “화면”을 누르면 그 시점 스크린샷이 열립니다. **통과한 테스트에도 동일하게 붙습니다.**

실패 항목은 **그 시점 화면이 리포트 파일 안에** 들어 있습니다. 링크나 별도 첨부 없이 리포트 하나만 전달해도 상대가 그대로 볼 수 있습니다 — 개발자에게 재현 요청을 넘길 때의 **왕복을 줄이기 위한 설계**입니다.

■ 상태를 네 가지로 분리한 이유

표시	뜻	취급
완료	검증을 통과	정상
실패	기대 결과와 다름 — 사유와 화면이 함께 제공	원인 확인
불안정	재시도해서 통과	초록이 아님 · 확인 필요
건너뛴	선행 실패 또는 중단으로 미실행	선행 결과부터 확인

왜 네 가지로 나눴나

“재시도해서 통과했으니 통과”로 넘기면 **flaky가 영원히 쌓입니다**. 불안정을 별도 상태로 분리해, flaky를 운이 아니라 원인 있는 결함으로 다루는 기준을 **도구 차원에서 고정**했습니다.

▼ 도구와 함께 만든 사용자 문서 · 결과

배포만으로는 아무도 쓰지 않는다

화면 순서를 그대로 따라가는 가이드를 **도구와 함께** 작성했습니다.

- **사용자의 언어로 쓴다** — “헤드리스”, “동시성 락” 대신 **“창 띄우기”, “지금 다른 실행이 진행 중입니다”**
- **막히는 화면을 미리 설명한다** — 실행이 차단되는 화면을 스크린샷과 함께 보여주고 **“이건 고장이 아닙니다”** 라고 먼저 말해 문의 자체를 줄임

- **문의가 들어올 지점을 먼저 답한다** — 실패가 많이 날 때 / 예상보다 오래 걸릴 때 / 리포트가 안 열릴 때를 트러블슈팅 섹션으로 선제 정리
- **문서를 화면에 종속시킨다** — 실제 화면 캡처를 그대로 담고, 화면이 바뀌면 문서도 함께 갱신하는 것을 전제로 구성

이 문서 자체가 산출물입니다. 도구를 만든 사람이 문서까지 책임지지 않으면 결국 문의가 다시 그 사람에게 돌아오고, 병목은 형태만 바뀌 유지됩니다.

결과

항목	Before	After
실행 주체	터미널 접근 가능한 QA 1인	개발·QA 팀원 누구나 (브라우저 주소 하나)
실행 준비	클론 · 의존성 설치 · 환경 변수 · 명령어 숙지	설치 0단계 · 명령어 0줄
동시 실행 사고	공유 계정 충돌로 양쪽 모두 실패	단독 실행 보장 + 실행자·경과 시간 노출
선행 조건	사용자가 의존 관계를 알고 직접 선택	러너가 자동 선택 · 포함 사유 표시
실행 중 이탈	세션이 끊기면 진행 상황 유실	재접속 시 로그·진행률 복원 (타인 실행 포함)
결과 회수	서버에 남고 개인 PC로 가져올 경로 없음	리포트 · 스크린샷 zip 원클릭 다운로드

이 작업에서 보여준 방식

- 검증을 나눠 갖는 구조를 만든다 — 품질 책임을 QA에 몰아두지 않고, 만든 사람이 직접 확인하게
- 자동화는 돌아가는 순간이 아니라, 남이 쓸 수 있을 때 끝난다 — 실행 UI와 사용자 문서를 자동화의 일부로 봄
- 막을 땀 이유를 함께 준다 — 없앨 수 없는 제약이라면 최소한 판단할 정보를 준다
- 도메인 지식이 필요한 부분은 도구가 대신 계산한다 — 선행 의존, 병렬 소요 시간
- 기준을 문서가 아니라 도구에 고정한다 — 불안정 상태 분리로 flaky 판정 기준을 강제

김상옥 · Kim Sang-Ok — SW QA Engineer · kx2471@gmail.com · github.com/kx2471

실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사·담당자명·버전·내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.

AI Agent — Regression TC Creator

제가 의무화한 Root Cause 기록을 읽어 회귀 TC를 설계하고, Google Sheets에 기록하고, 티켓에 코멘트와 필드까지 연결하는 Claude 기반 Agent입니다. **정책을 만든 사람이 그 정책이 남긴 데이터를 쓰는 도구를 직접 만들었습니다.**

PROBLEM

RCA 기록은 쌓이는데 활용이 안 되는 자산이 될 위험이 있었고, 회귀 TC 설계·기록·연결은 매 스프린트 반복되는 수작업이었습니다.

ACTION

MCP 도구 조합 · 3단계 게이팅 · 안전장치를 포함한 시스템으로 구현했습니다.

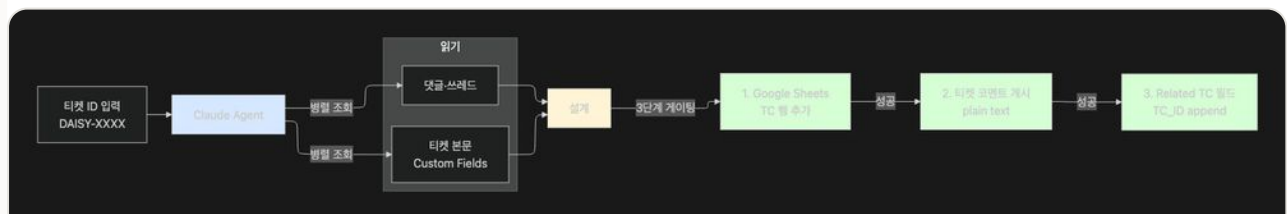
RESULT

티켓 ID 한 줄이면 TC 설계·시트 기록·티켓 연결이 자동으로 끝납니다. 타 팀이 사내 업무 개선 사례로 공유했습니다.

▼ 왜 만들었는가 · 동작 흐름

정책이 남긴 데이터를 쓰는 도구

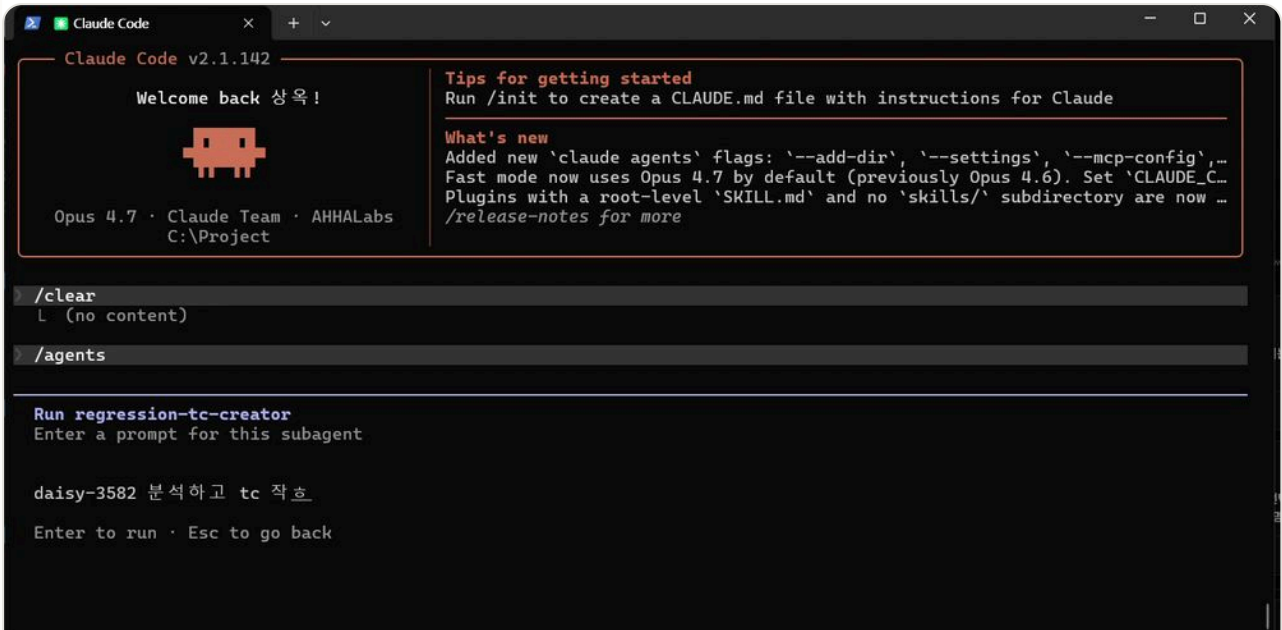
QA 워크플로우를 설계하면서 **버그 티켓의 Root Cause · Corrective Action**을 기록하도록 의무화했습니다. 그런데 이 정보는 *기록만 되고 활용이 안 되는 자산*이 될 위험이 있었습니다 — 매 스프린트마다 버그 티켓을 일일이 읽고, 회귀 TC를 손으로 설계하고, 시트에 기록하고, 원본 티켓에 연결하는 과정이 **단순 반복의 연속**이었습니다.



설계 단계에서 그린 실제 동작 흐름. 티켓 본문·Custom Fields와 댓글·쓰레드를 병렬 조회한 뒤 설계하고, 세 개의 쓰기를 순차 게이팅으로 묶었다.

세 가지 쓰기 작업은 **앞 단계가 성공해야 다음이 진행**됩니다. 일부만 게시되는 **불완전 상태**를 원천적으로 차단하기 위함입니다.

호출은 한 줄이면 끝난다



서브에이전트로 등록해 두고 티켓 ID만 넘긴다 — “daisy-3582 분석하고 tc 작성해줘”.

에이전트를 **서브에이전트로 등록**해 두고, 티켓 ID 하나만 자연어로 넘기면 됩니다. 별도 설정이나 옵션 플래그 없이 **기본 시트에 기록되도록** 기본값을 잡아, 쓰는 사람이 매번 결정할 것을 없앴습니다.

왜 이렇게

도구의 채택률은 기능 수가 아니라 **처음 쓸 때의 마찰**에서 갈립니다. “무엇을 설정해야 하지?”가 한 번이라도 떠오르면 사람들은 그냥 손으로 합니다.

▼ 설계 원칙 — QA 엔지니어의 사고 과정을 명문화

RCA 기반 · 5축 스캔 · 양보다 질

1. RCA 기반 TC 설계

단순히 티켓을 요약하는 게 아니라, **과거 실패 모드를 재현하는** 일을 합니다.

- Root Cause가 명시된 경우 → 그 실패 모드를 **재현하는 명시적 케이스** 설계
- Corrective Action이 실제로 막거나 처리하는지 확인하는 **회귀 케이스** 추가
- 같은 근본 원인의 영향을 받을 수 있는 **인접 경계 조건**도 함께 설계

2. 5축으로 시나리오 스캔

축	명세 / 예시
Happy-path	의도대로 동작하는 주요 end-to-end 시나리오
입력/데이터 변형	경계값, 일반값, 의미 있는 조합
에러/실패 조건	잘못된 입력, 누락 데이터, 타임아웃, 권한, 연동 실패
연동/외부 시스템	다른 서비스, 모델 추론, 스토리지, 백엔드 응답 기반 UI 갱신
Frequency/반복	빈번 실행, 배치/스케줄, 피크 타임 시 안정성

3. 양보다 질 — 역지로 채우지 않기

에이전트가 **형식적으로 TC를 짚어내는 것을 막기 위해** 출력량 규칙을 명확히 정의했습니다.

- **기본 목표:** 티켓당 3개 (happy-path 1 + 핵심 회귀 1 + 인접/예외 1)
- **축소:** 단순 버그(텍스트 수정, 좁은 정규식 보정)는 1~2개로 마무리 가능
- **확장:** Corrective Action이 여러 기능 영역에 걸치거나, Frequency가 Always이고 부하 케이스가 본질일 때
- **금지:** 형식적 happy-path나 의미 없는 변형으로 빈자리 채우기

시트에 넘기기 애매한 보조 검증 아이디어는 시트가 아닌 코멘트의 “추가로 검증되어야 할 점” 블록에 담아 시트와 코멘트의 역할을 분리했습니다.

3단계 쓰기 구조와 실제 결과

Sheets → 코멘트 → Related TC

Step 1. Google Sheets 기록 (정본)

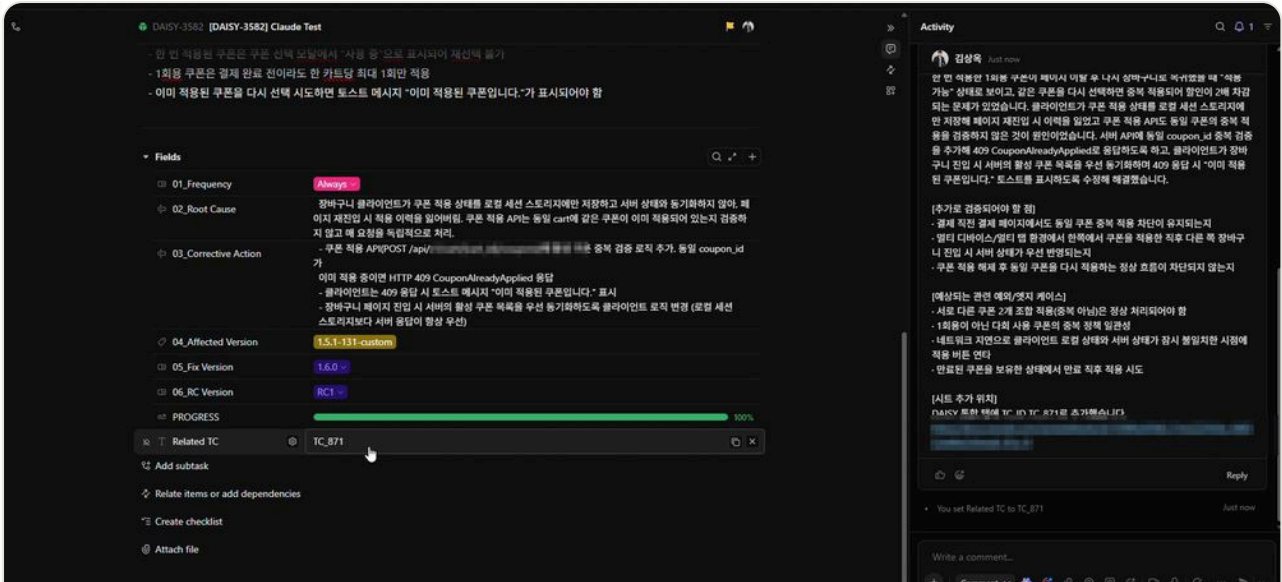
14컬럼 양식으로 시트 마지막 행 다음에 직접 기록합니다. Agent가 추가한 행은 **주황색 배경**으로 칠해 사람이 작성한 TC와 즉시 구분되도록 했고, `wrapStrategy: WRAP` 을 적용해 셀 안 줄바꿈이 실제로 보이도록 했습니다.

TC_ID	타입	기능영역	세부영역	모델타입	TC명	사전조건	테스트 절차	기대결과	레이블
TC_871					장바구니 - 동일 1회용 주문 중복 적용 차단 및 페이지 재전입 시 적용 상태 동기화	일반 회원으로 로그인한 상태 장바구니에 상품 1개 이상 담겨 있음 사용자 쿠폰함에 1회용 "5,000원 할인 쿠폰" 1개가 발급되어 있음	1) 장바구니 페이지 진입 2) 주문 적용 버튼 클릭하여 주문 선택 옵션 열기 3) "5,000원 할인 쿠폰" 선택 후 적용 버튼 클릭 4) 결제로 진행하지 않고 카테고리 또는 홈으로 이동 5) 다시 장바구니 페이지로 복귀하여 주문 적용 상태 확인 6) 주문 적용 버튼을 다시 클릭하여 주문 선택 옵션 재전입 7) 이미 적용된 "5,000원 할인 쿠폰"의 상태 확인 및 재선택 시도	- 5단계: 장바구니 복귀 시 "5,000원 할인 쿠폰"이 이미 적용된 상태로 유지되고 주문 요약에 "할인 5,000원"으로 표시됨 - 6단계: 주문 선택 옵션에서 해당 쿠폰이 "사용 중" 상태로 표시되고 재선택이 차단됨 - 7단계: 강제로 재선택을 시도하면 토스트 메시지 "이미 적용된 쿠폰입니다."가 노출되고 중복 적용되지 않음 - 주문 요약 총 할인 금액이 -10,000원이 아닌 -5,000원으로 유지됨	Regression

Step 1 — 마지막 행 다음에 회귀 TC가 추가된다. Agent가 넣은 행은 주황색 배경이라 사람이 쓴 TC와 즉시 구분된다. (TC_871 외 실제 TC는 비식별 처리)

Step 2 · 3. 코멘트 게시 → Related TC 연결

TC 표는 코멘트에 넣지 않고 **TC는 시트가 정보**이라는 원칙을 지킵니다. 코멘트에는 간단 분석 · 추가 검증 포인트 · 관련 옛지 케이스 · 시트 추가 위치만 들어갑니다. 생성한 TC_ID는 티켓의 Related TC 필드에 **기존값 보존한 채 append**해 양방향 추적성이 자동으로 남습니다.



Step 2:3 — 티켓에 분석 코멘트가 게시되고(오른쪽), Related TC 필드에 TC_871이 연결된다(왼쪽 하단). Agent가 읽는 입력인 Root Cause · Corrective Action 필드도 같은 화면에 보인다.

이는 제가 설계한 QA 워크플로우의 세 번째 원칙(**Root Cause 의무화**)에서 끝나지 않고, 그 기록이 검증 자산으로 이어지는 고리를 만드는 단계입니다. → 01 프로세스 설계

▼ 안전장치와 구현 중 마주친 문제들

근거 없이 쓰지 않는 구조

안전장치

- **미리보기 모드** — “초안만 보여줘”라고 하면 시트·코멘트·필드 갱신을 모두 생략하고 화면에만 출력
- **부분 거부 수용** — “시트에 쓰지 마” 같은 지시를 개별 처리, 거부된 단계 이후는 자동으로 함께 생략
- **fallback 시트 탐색** — 기본 시트 접근 불가 시에만 후보를 제시, 확정 안 되면 쓰기 전체 보류
- **필드 있을 때만 갱신** — Related TC 필드가 없으면 생성하지 않고 건너뛰기
- **쓰기 후 검증** — 같은 범위를 다시 읽어 실제 들어간 행 번호·TC_ID 범위를 확인

구현 중 마주친 실제 문제들



Sheets API가 잘못된 위치에 행을 넣는 문제

드롭다운·조건부 서식이 걸린 빈 영역이 있으면 API가 그 영역까지를 "테이블"로 인식해 **마지막 데이터 행에서 10~20행 떨어진 곳에 행을 박아버렸습니다.**

→ `add_rows` 를 포기하고, 마지막 데이터 행 + 1부터 시작하는 **명시적 셀 범위**를 계산해 직접 쓰도록 설계 변경.



코멘트가 마크다운을 렌더링하지 않는 문제

마크다운으로 쓰면 `**강조**` · `##` 헤더 · `-` 볼릿 이 모두 *글자 그대로* 노출되었습니다.

→ 마크다운 전면 금지로 규칙을 바꿔, 제목은 대괄호, 볼릿은 가운데점, 강조는 `ALL`

CAPS: 패턴으로 **plain text**에서도 시각 구조가 남도록 재설계.



TC_ID 체계를 "추측"하지 않음

시트 마지막 행의 TC_ID를 **먼저 읽고**, 그 형식을 유지한 채 일련번호만 증가시킵니다. 순수 숫자면 +1, 접두어가 있으면 접두어 유지. **Agent가 출처·형식을 마음대로 만들지 않도록 제약**하는 것이 핵심이었습니다.



이 작업이 보여주는 것

- 정책 설계자와 도구 빌더가 같은 사람 — RCA 의무화 정책을 만든 사람이 그 데이터를 쓰는 도구를 만든 일관성
- AI를 도구가 아닌 워크플로우 구성원으로 설계 — 프롬프트 래퍼가 아니라 게이팅·안전장치를 포함한 시스템
- QA 사고 과정 자체를 명문화 — 5층 스캔, RCA 기반 설계, 양보다 질 원칙

▼ 사내 확산 — 내 도구에서 팀의 사례로

타 팀이 개선 사례로 공유

만들어 쓰는 것과 **조직이 받아들이는** 것은 다른 문제입니다. 이 에이전트는 제가 홍보한 것이 아니라 **다른 팀이 사내 개선 사례로 소개**하면서 알려졌습니다.



기획) # general

5월 18일, 오후 4:39

업무 개선 사례를 공유합니다🌸

최근 저희 팀에서 클로드 유료버전을 사용하면서 업무 병목을 해결하기 위한 아이디어를 같이 나누고 있는데요, 그 첫번째로, QA 티켓 발행 슬랙봇을 만들기도 했던 @김상옥(마케팅/기획 - QA) 님의 [Regression TC Creator Agent 사례](#)를 소개합니다.

앞으로도 좋은 사례 함께 나눠요~🥰

[Claude Agent - Regerssion TC Creator](#)

1. 이 에이전트로 할 수 있는 일 ClickUp 티켓 ID를 주면 다음 3가지가 자동으로 처리됩니다.
Google Sheets에 회귀 TC 행 추가 (주황색 배경으로 표시) ClickUp 티켓에 분석 코멘트 게시 티켓의 Related TC 커스텀 필드에 TC_ID 자동 연결 2. 기본 사용법 가장 간단한 호출 DAISY-3535 분석해서 회귀 TC 만들어줘 이 한 줄이면 끝납니다. 별도 설정 없이 기본 시트(DAISY TC_신규 > DAISY 통합 탭)에 기록되...

[자세히 표시](#)

in Docs (2025.04~) 김상옥

[View doc](#)

봇이 추가한 ClickUp

👁 9 🗨 12 📄 8 📊 7 📅 6 📌 7 👍 10 🍌 4 😊

사내 #general 채널에 "업무 개선 사례"로 공유된 Regression TC Creator. 60개 이상의 리액션이 달렸다. (공유자 정보는 비식별 처리)

확산을 염두에 두고 만든 것들

- **한 줄 호출** — 설정·플래그 없이 기본 시트에 기록되도록 기본값을 잡음
- **결과가 눈에 보이게** — 주황색 행, 티켓 코멘트, 필드 연결. "돌렸는데 뭐가 달라졌는지 모르겠다"가 없도록
- **사용 가이드 동봉** — 할 수 있는 일 · 기본 사용법 · 옵션을 사내 문서로 함께 배포

이 반응이 의미하는 것

- 도구가 **QA 팀 밖에서도 이해 가능한 형태**였다는 것
- AI 활용이 개인 실험이 아니라 **팀이 따라 할 수 있는 사례**가 됐다는 것
- 같은 맥락에서 **QA Request** 봇도 함께 언급됨 — 단발성이 아니라 **반복적으로 도구를 만들어 온 사람**으로 인식됨

돌아보며

좋은 도구를 만드는 것보다 **남이 쓰게 만드는 것이 훨씬 어렵다**는 걸 테스트 러너와 이 에이전트에서 반복해서 배웠습니다. 그래서 이제는 도구를 만들 때 **사용 가이드와 첫 실행 경험을 같은 작업**으로 취급합니다.

실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사·담당자명·버전·내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.

검증 실무 — 산출물 · API 검증 · 업무 도구

SWR · TC · Defect · Xray 추적성 같은 QA 표준 산출물과, UI 뒤를 직접 두드리는 API 검증, 반복 업무를 줄이려고 만든 유틸리티들입니다.

PROBLEM 요구사항-TC-결함이 따로 놓였고, UI에서 막히는 값이 서버에서도 막히는지는 확인되지 않았습니다.	ACTION 추적성 체계를 세우고, API 레벨로 내려가 검증하고, 반복은 도구로 줄였습니다.
RESULT 요구사항 커버리지와 결함 추적성 확보, 서버 측 검증 부재 발견·개선 주도.	

▼ QA 산출물 — 요구사항에서 결함까지 추적 가능하게

SWR · TC · Defect · Xray

Test Case 설계 기법

기법 정의보다 실제로 무엇을 만들어 관리했는지를 남깁니다.

기법	적용한 방식	남긴 것
요구사항 기반	요구사항 문서를 항목 단위로 쪼개 빠진 기능이 없는지 대조	Jira 등록 후 요구사항 ID와 매핑
동등 클래스 분할	입력값 범위·조건을 분석해 유효/무효 클래스로 나누고 대표값 선정	필드별 클래스 정의 — 최소 TC로 최대 결함
경계값 분석	각 필드의 최솟값·최댓값과 직전/직후를 도출. off-by-one 방지	min · min-1 · max · max+1
오류 추정	과거 결함 이력 · 복잡한 로직 · 사용자 오용 시나리오로 의심 영역을 좁힘	정형 기법으로 안 잡히는 옛지 케이스

기법을 나눠 관리한 이유 — 어떤 기법으로 만든 TC인지가 남아 있어야 커버리지의 성격을 말할 수 있습니다.

“TC 300개”보다 “요구사항 전 항목 대조 + 경계값 조합 + 과거 결함 기반 의심 영역”이 훨씬 정확한 진술이고, 회귀 대상을 고를 때도 이 축이 기준이 됩니다.

Xray 기반 추적성

요구사항과 TC의 연결로 추적성을 확보하고, 결함 관리 연동으로 양방향 추적이 가능합니다.

- **타입·정밀도 경계를 그대로 통과시켜 본다** — 정수 필드에 실수를, 불리언에 문자열을 넣었을 때 서버가 무엇을 저장하는지
- **도달하기 어려운 상태를 만들어 놓고 시작한다** — 배포 상태 같은 값을 직접 PUT으로 세팅하면 UI로 몇 단계 거쳐야 하는 전제 조건을 한 번에 구성
- **거부돼야 할 요청을 만들어 보낸다** — 인증 헤더를 빼거나 바꿔서, 통과하는 경우가 아니라 **막혀야 하는 경우**를 확인

발견 사례 — 프론트만 막고 있던 유효성 검증

진행

- DevTools로 회원가입·회원정보 수정 요청 패킷 추출
- 파라미터 구조와 인증 방식 분석
- Postman·cURL로 형식 위반 값과 `<script>` 태그를 삽입해 전송
- 응답만 보지 않고 **DB 저장 여부와 관리자 페이지 반영까지 추적**

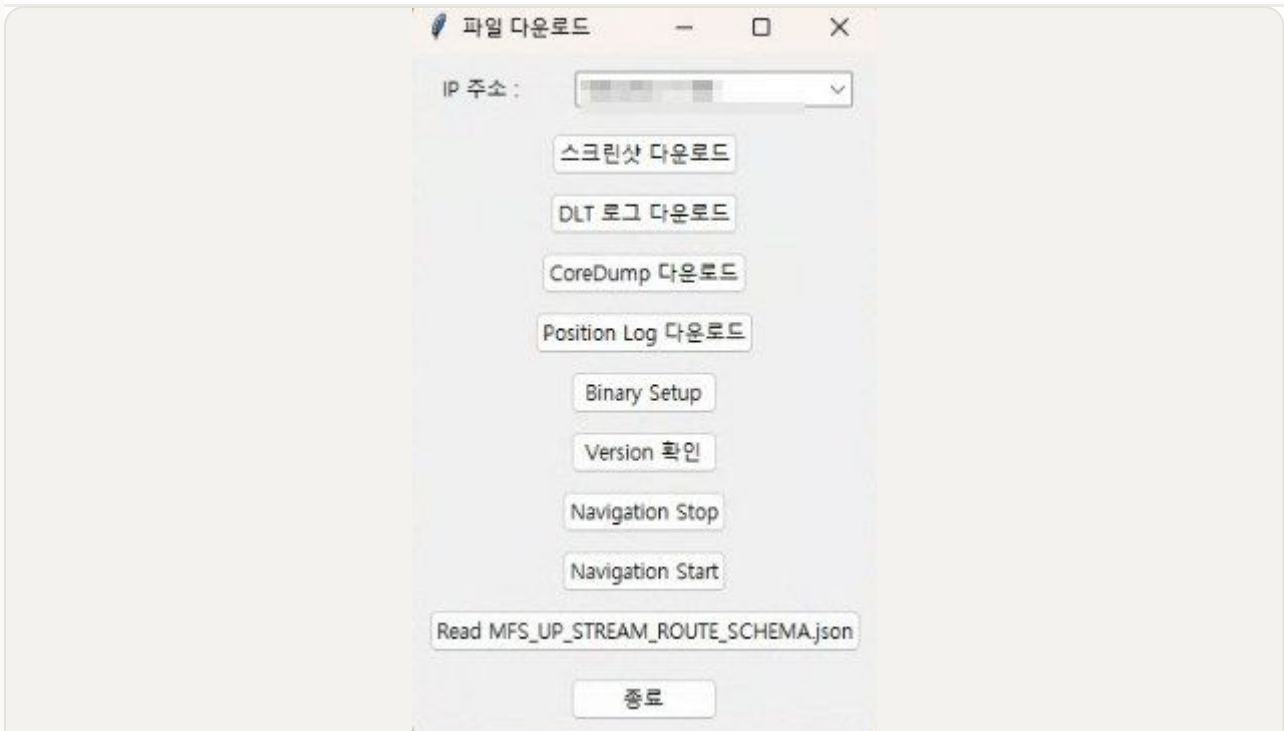
발견

- **서버 측 입력값 검증 부재** — 프론트엔드 검증만 존재
- 형식·규칙을 위반한 데이터가 **그대로 DB에 저장**
- 주입한 스크립트가 **관리자 페이지에서 렌더링되는 경로 확인** → XSS

보고

- **재현 절차를 그대로 옮길 수 있게 정리**
- **스크린샷과 패킷 로그를 증거로 확보**
- UI 결함이 아니라 **서버 검증 로직 부재**로 원인 규정
- 개선 방향까지 제안해 **수정 주도**

화면 기준 테스트만 했다면 끝까지 발견되지 않았을 결함입니다. 그 뒤로 품이 있는 기능은 **UI 통과와 API 통과를 별도의 검증 항목으로 나눠 씁니다.**



IP를 고르고 버튼 하나로 로그·스크린샷·CoreDump를 받아온다. 서비스 중단/재시작과 바이너리 배포까지 같은 창에서 처리. (장비 IP 비식별)

종합추출도구

Python · paramiko · Tkinter

원격 서버에 SSH로 접속해 로그·스크린샷·CoreDump·Binary를 자동 수집하는 GUI 도구. 서비스 중단/재시작과 바이너리 배포까지 같은 창에서 처리합니다.

KPI Calculator

Python · re · threading

DLT 로그에서 KPI 이벤트를 정규식으로 파싱해 구간별 시간 차를 계산. **수작업 대비 검증 리소스 30% 절감**에 기여했습니다.

Positioning Viewer

Python · pandas · folium

GNSS 로그에서 좌표를 추출·검증해 CSV로 저장하고, 지도 위에 주행 경로를 폴리라인으로 시각화합니다.

Clipboard Translate

Python · Tesseract OCR

클립보드 이미지·텍스트를 실시간 감지해 OCR + 번역. OCR 줄바꿈을 문장 종결 기준으로 병합하고 기술 문서용 **명사형 종결로 자동 변환**합니다.

공수표 알리미

Google Apps Script x Slack

“매달 잊지 말고 공유해야 하는 링크”를 사람의 기억에서 시스템으로 옮겼습니다. **링크 누락 시 경고, 월말 사전 점검**까지 포함 — 실패 케이스를 설계에 넣었습니다.

Selenium 반응형 UI 도구

Python · Selenium · Tkinter

여러 Breakpoint에서 자동 로그인 후 페이지를 순회하며 스크린샷을 일괄 캡처. GitHub Releases로 실행 파일을 배포해 **비개발자도 쓸 수 있게** 했습니다.

공통점

전부 “같은 일을 두 번 하고 있다”는 자각에서 출발했습니다. 그리고 만들 때는 실패 케이스(링크 누락, 경로 못 찾음, 무한 루프)를 설계 범위에 넣었습니다.

김상옥 · Kim Sang-Ok — SW QA Engineer · kx2471@gmail.com · github.com/kx2471

실제 업무 경험을 바탕으로 재구성·가공되었으며, 회사 기밀 보호를 위해 고객사 · 담당자명 · 버전 · 내부 주소 등 민감 정보는 익명화하거나 비식별 처리했습니다.